

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI STACK PADA JAVA

Disusun oleh:

Rahmat Ananda Nazar

2511532008

Dosen Pengampu : Dr. Wahyudi S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa berkat rahmat dan karunianya penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 3 ini dengan baik. Pada laporan praktikum pekan satu ini akan membahas mengenai penerapan Struktur data Stack pada Java.

Penulis menyadari bahwa laporan ini tidak terlepas dari kesalahan dan kekurangan baik dari penulisan dan pembahasan. Oleh karena itu penulis ingin mengucapkan terimakasih kepada dosen pengampu Bapak Dr. Wahyudi, S.T M.T. dan asisten praktikum Muhammad Galid Averro yang telah memberikan arahan selama proses praktikum dan proses penyusunan laporan praktikum. Penulis berharap laporan ini dapat bermanfaat bagi pembaca dalam memahami Struktur data Stack pada java

DAFTAR ISI

DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Class LatihanStack	2
2.1.1 Kode Program dan Output	2
2.1.2 Penjelasan Program	2
2.2 Class NilaiMaksimum	3
2.2.1 Kode Program dan Output	3
2.2.2 Penjelasan Program	3
2.3 SiswaStack	4
2.3.1 Kode Program dan Output	4
2.3.2 Penjelasan Program	5
2.4 StackArray dan StackArray Driver	6
2.4.1 Kode Program dan output	6
2.4.2. Penjelasan Program	7
2.5 StackPostFix	8
2.5.1 Kode Program dan Output	8
2.5.2 Penjelasan Program	9
BAB III PENUTUP	10
3.1 Kesimpulan	10
DAFTAR PUSTAKA	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, penyusunan data secara efisien merupakan aspek krusial untuk membangun sistem yang terstruktur dan mudah dikelola. Salah satu konsep fundamental dalam mata kuliah Struktur Data adalah Abstract Data Type (ADT), Praktikum ini berfokus pada penerapan konsep lanjutan untuk ADT melalui pemahaman struktur data STACK. Melalui implementasi ini, diharapkan praktikan dapat memahami cara menggunakan struktur data Stack pada program.

1.2 Tujuan Praktikum

Implementasi Struktur data Stack agar mampu menerapkannya pada program untuk membuat sistem yang relevan dengan kebutuhan.

1.3 Manfaat Praktikum

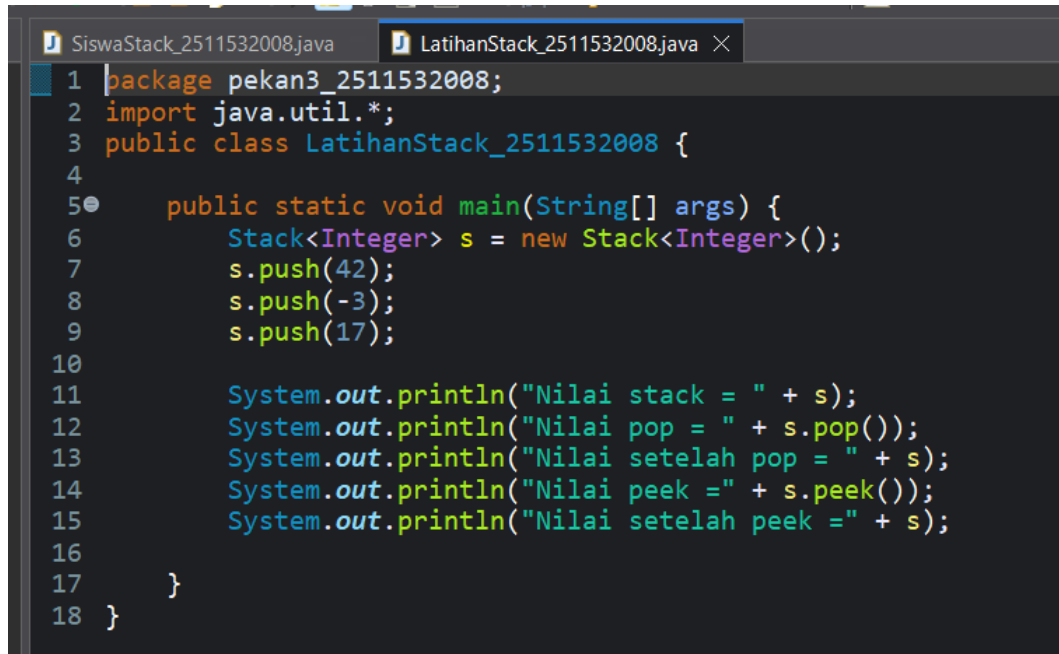
Praktikum ini memberikan manfaat pemahaman teoretis melalui penerapan langsung struktur data Stack pada program. Melalui kegiatan ini, praktikan dapat mengasah keterampilan teknis dalam membangun struktur data yang rapi menggunakan kelas ADT, mengelola alur program yang dinamis dengan Stack, praktikum ini melatih kemampuan pemecahan masalah (problem solving) dan logika berpikir sistematis saat mengintegrasikan berbagai fungsi ke dalam satu aplikasi yang fungsional dan terstruktur.

BAB II

PEMBAHASAN

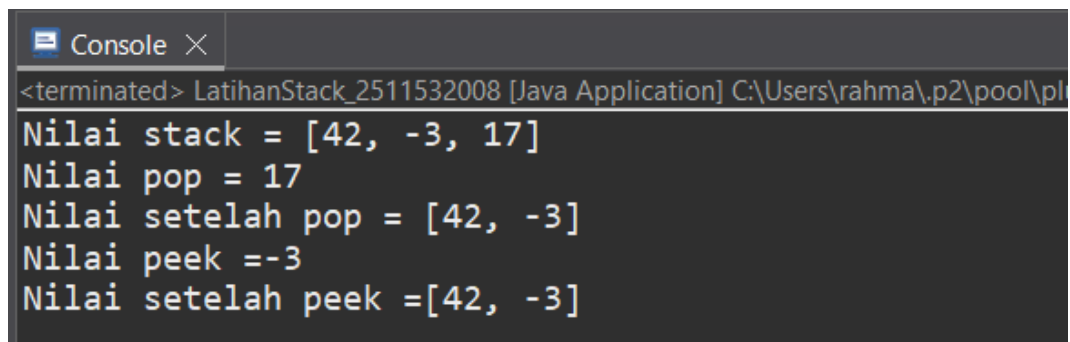
2.1 Class LatihanStack

2.1.1 Kode Program dan Output



```
1 package pekan3_2511532008;
2 import java.util.*;
3 public class LatihanStack_2511532008 {
4
5     public static void main(String[] args) {
6         Stack<Integer> s = new Stack<Integer>();
7         s.push(42);
8         s.push(-3);
9         s.push(17);
10
11         System.out.println("Nilai stack = " + s);
12         System.out.println("Nilai pop = " + s.pop());
13         System.out.println("Nilai setelah pop = " + s);
14         System.out.println("Nilai peek = " + s.peek());
15         System.out.println("Nilai setelah peek =" + s);
16
17     }
18 }
```

Gambar 2.1



```
<terminated> LatihanStack_2511532008 [Java Application] C:\Users\rahma\p2\pool\pl
Nilai stack = [42, -3, 17]
Nilai pop = 17
Nilai setelah pop = [42, -3]
Nilai peek = -3
Nilai setelah peek =[42, -3]
```

Gambar 2.2

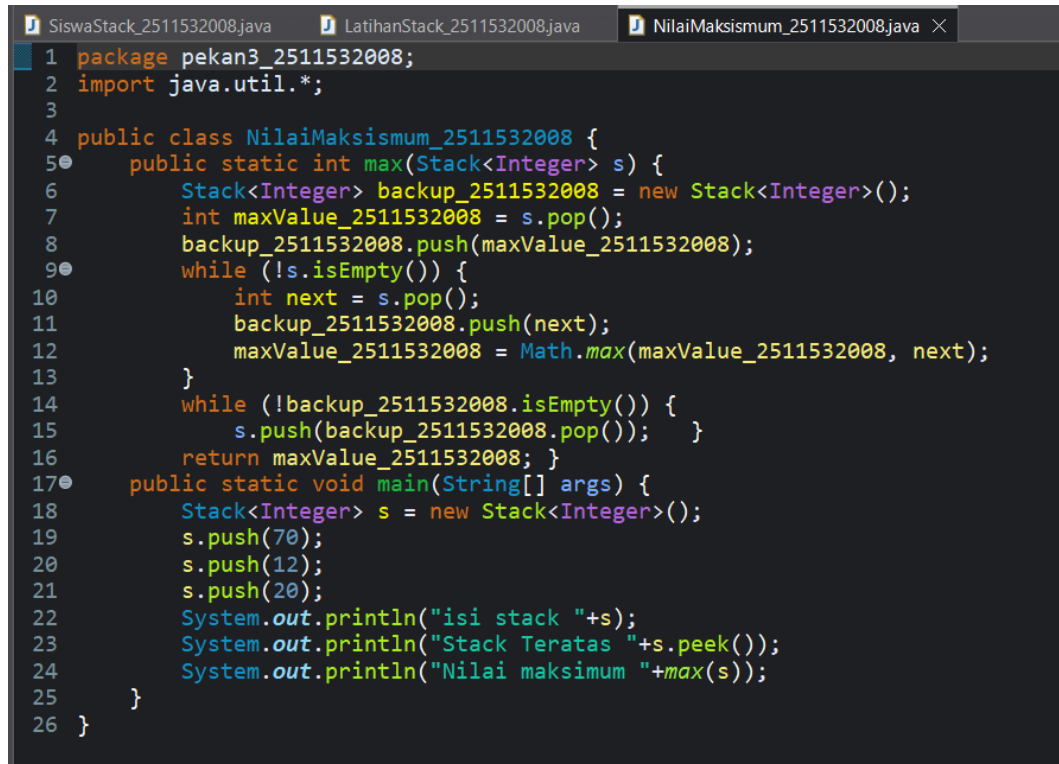
2.1.2 Penjelasan Program

Dalam program ini di inisialisasikan Struktur data Stack menggunakan sintaks `Stack s = new Stack ()` untuk kemudian kita bisa melakukan push 3 nilai untuk dimasukan kedalam stack bisa dilihat pada baris pertama output. Kemudian menggunakan sintaks `pop()` untuk menghapus elemen paling atas pada stack dan

mereturn valuenya, sementara peek() mereturn nilai paling atas pada stack namun tidak menghapus valuenya.

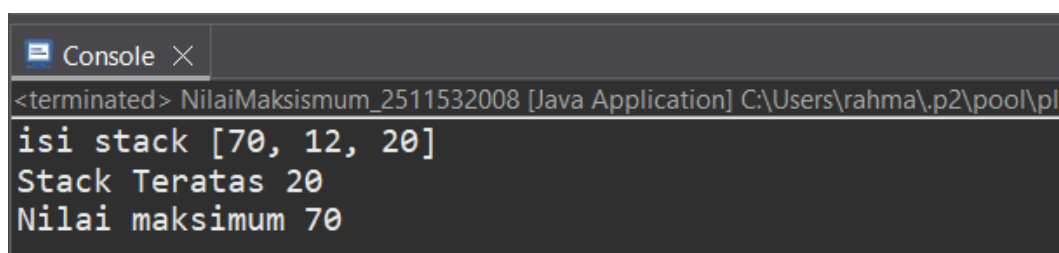
2.2 Class NilaiMaksimum

2.2.1 Kode Program dan Output



```
1 package pekan3_2511532008;
2 import java.util.*;
3
4 public class NilaiMaksimum_2511532008 {
5     public static int max(Stack<Integer> s) {
6         Stack<Integer> backup_2511532008 = new Stack<Integer>();
7         int maxValue_2511532008 = s.pop();
8         backup_2511532008.push(maxValue_2511532008);
9         while (!s.isEmpty()) {
10             int next = s.pop();
11             backup_2511532008.push(next);
12             maxValue_2511532008 = Math.max(maxValue_2511532008, next);
13         }
14         while (!backup_2511532008.isEmpty()) {
15             s.push(backup_2511532008.pop());
16         }
17         return maxValue_2511532008;
18     }
19     public static void main(String[] args) {
20         Stack<Integer> s = new Stack<Integer>();
21         s.push(70);
22         s.push(12);
23         s.push(20);
24         System.out.println("isi stack "+s);
25         System.out.println("Stack Teratas "+s.peek());
26         System.out.println("Nilai maksimum "+max(s));
27     }
28 }
```

Gambar 2.3



```
<terminated> NilaiMaksimum_2511532008 [Java Application] C:\Users\rahma\p2\pool\plu
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 2.4

2.2.2 Penjelasan Program

Pada program ini kita akan mengambil nilai max pada stack menggunakan fungsi max() untuk mencari nilai paling tinggi yg ada pada Stack

2.3 SiswaStack

2.3.1 Kode Program dan Output

```
1 package pekan3_2511532008;
2
3 import java.util.*;
4
5 class Siswa_2511532008 {
6     String nama; int nim;
7     public Siswa_2511532008(String nama, int nim) {
8         this.nama = nama;
9         this.nim = nim; }
10    public String toString() {
11        return "Nim: " + nim + ", Nama: " + nama; }
12 }
13 public class SiswaStack_2511532008 {
14     private ArrayList<Siswa_2511532008> stack;
15     public SiswaStack_2511532008() {
16         stack = new ArrayList<>();}
17     public void push(Siswa_2511532008 mhs) {
18         stack.add(mhs); }
19     public Siswa_2511532008 pop() {
20         if (!isEmpty()) {
21             return stack.remove(stack.size()-1); }
22         return null; }
23     public Siswa_2511532008 peek() {
24         if (!isEmpty()) {
25             return stack.get(stack.size()-1);}
26         return null; }
27     public boolean isEmpty() {
28         return stack.isEmpty(); }
29     public void tampilkanSiswa_2511532008() {
30         for (int i = stack.size()-1; i >= 0; i--) {
31             System.out.println(stack.get(i));}
32     public static void main (String[]args) {
33         SiswaStack_2511532008 StudentStack = new SiswaStack_2511532008();
34         Siswa_2511532008 mhs1 = new Siswa_2511532008("Ali", 1);
35         Siswa_2511532008 mhs2 = new Siswa_2511532008("Boby", 2);
36         Siswa_2511532008 mhs3 = new Siswa_2511532008("Charles", 3);
37         Siswa_2511532008 mhs4 = new Siswa_2511532008("Rahmat", 4);
38         StudentStack.push(mhs1);
39         StudentStack.push(mhs2);
40         StudentStack.push(mhs3);
41         StudentStack.push(mhs4);
42         System.out.println("Siswa di dalam Stack ");
43         StudentStack.tampilkanSiswa_2511532008();
44         System.out.println("Siswa teratas " + StudentStack.peek());
45         System.out.println("Mengeluarkan siswa teratas dari stack : " + StudentStack.pop());
46         System.out.println("Daftar siswa setelah di pop : ");
47         StudentStack.tampilkanSiswa_2511532008();
48     }
49 }
50 }
```

Gambar 2.5

```
Console X
<terminated> SiswaStack_2511532008 [Java Application] C:\Users\rahma\p2\pool\plugins\org.eclipse.justi.ope
Siswa di dalam Stack
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
Mengeluarkan siswa teratas dari stack : Nim: 3, Nama: Charles
Daftar siswa setelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

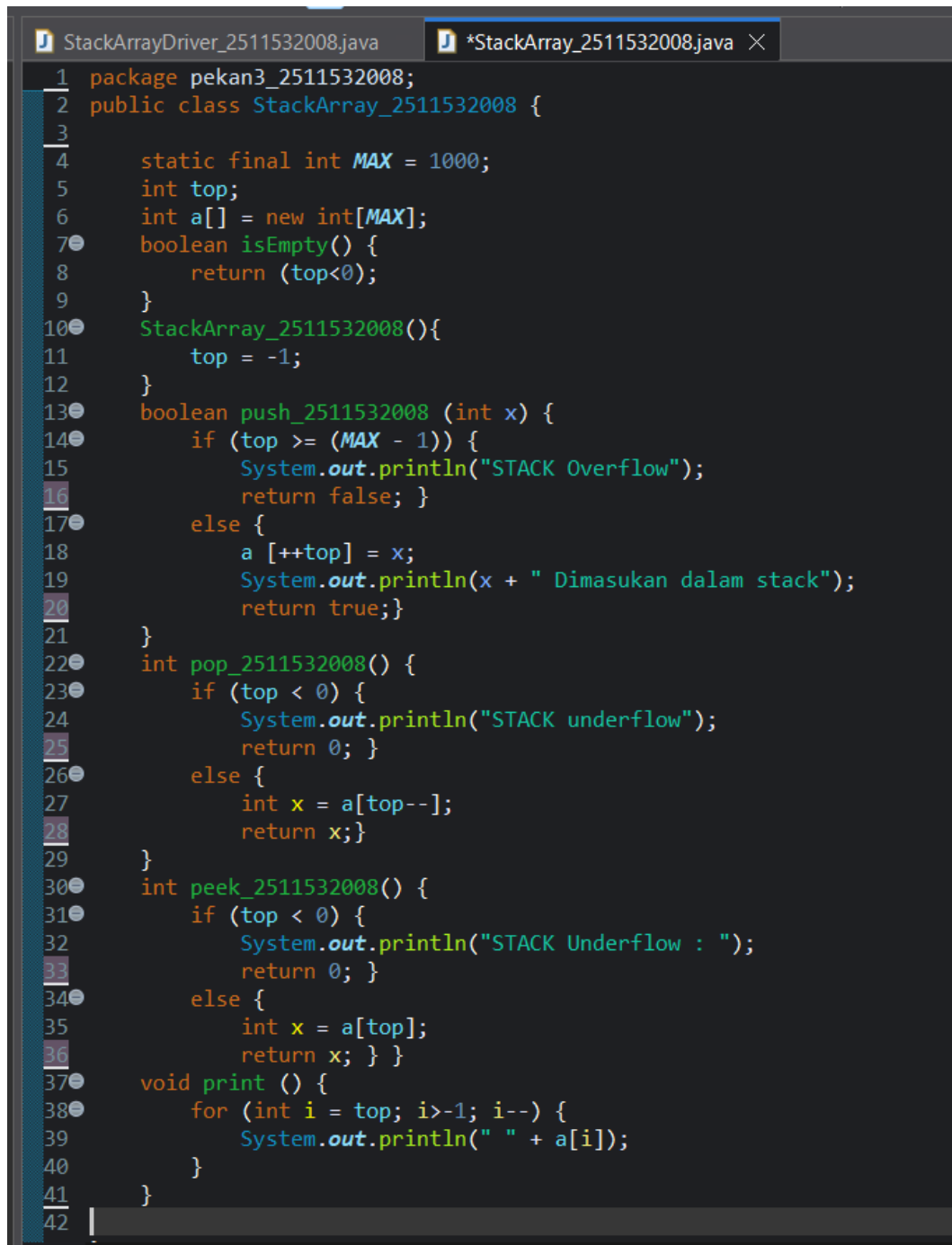
Gambar 2.6

2.3.2 Penjelasan Program

Implementasi Custom Stack berbasis ArrayList ini mendemonstrasikan penerapan prinsip LIFO (Last In First Out) dalam mengelola objek Siswa secara dinamis dan terstruktur. Dengan memanipulasi indeks terakhir sebagai puncak tumpukan melalui metode push, pop, dan peek, sistem ini menjamin keamanan operasional lewat validasi isEmpty serta keterbacaan data melalui override toString. Penggunaan iterasi terbalik dalam menampilkan data memastikan representasi logis tumpukan tetap akurat, sekaligus membuktikan fleksibilitas Java dalam menangani objek kompleks secara sistematis.

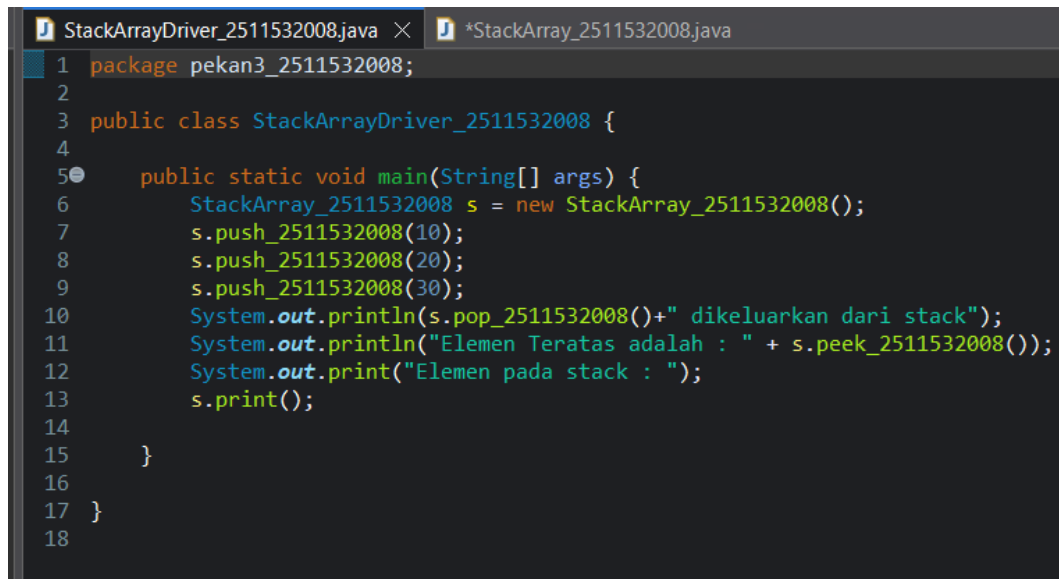
2.4 StackArray dan StackArray Driver

2.4.1 Kode Program dan output

The image shows a screenshot of a Java IDE with two tabs open. The active tab is 'StackArrayDriver_2511532008.java', which contains the driver code. The other tab is '*StackArray_2511532008.java'. The code in the active tab is as follows:

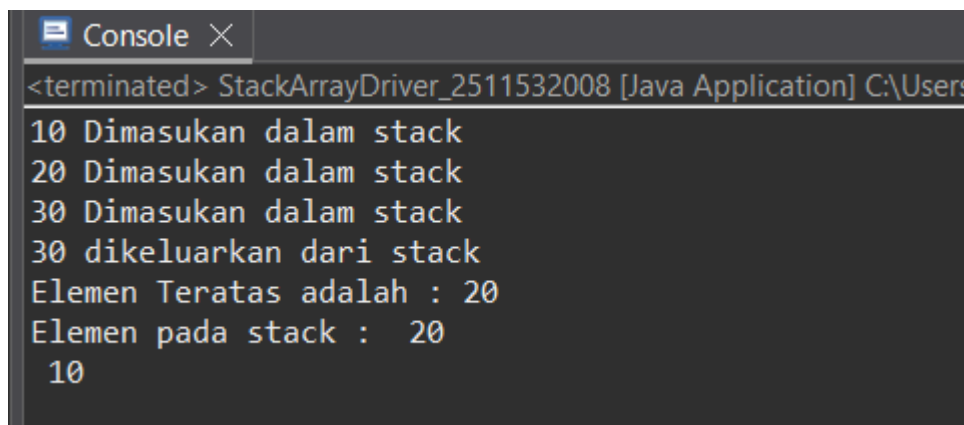
```
1 package pekan3_2511532008;
2 public class StackArray_2511532008 {
3
4     static final int MAX = 1000;
5     int top;
6     int a[] = new int[MAX];
7     boolean isEmpty() {
8         return (top<0);
9     }
10    StackArray_2511532008(){
11        top = -1;
12    }
13    boolean push_2511532008 (int x) {
14        if (top >= (MAX - 1)) {
15            System.out.println("STACK Overflow");
16            return false; }
17        else {
18            a[++top] = x;
19            System.out.println(x + " Dimasukan dalam stack");
20            return true;}
21    }
22    int pop_2511532008() {
23        if (top < 0) {
24            System.out.println("STACK underflow");
25            return 0; }
26        else {
27            int x = a[top--];
28            return x;}
29    }
30    int peek_2511532008() {
31        if (top < 0) {
32            System.out.println("STACK Underflow : ");
33            return 0; }
34        else {
35            int x = a[top];
36            return x; } }
37    void print () {
38        for (int i = top; i>-1; i--) {
39            System.out.println(" " + a[i]);
40        }
41    }
42 }
```

Gambar 2.7



```
1 package pekan3_2511532008;
2
3 public class StackArrayDriver_2511532008 {
4
5     public static void main(String[] args) {
6         StackArray_2511532008 s = new StackArray_2511532008();
7         s.push_2511532008(10);
8         s.push_2511532008(20);
9         s.push_2511532008(30);
10        System.out.println(s.pop_2511532008()+" dikeluarkan dari stack");
11        System.out.println("Elemen Teratas adalah : " + s.peek_2511532008());
12        System.out.print("Elemen pada stack : ");
13        s.print();
14    }
15 }
16
17 }
18
```

Gambar 2.8



```
<terminated> StackArrayDriver_2511532008 [Java Application] C:\Users
10 Dimasukan dalam stack
20 Dimasukan dalam stack
30 Dimasukan dalam stack
30 dikeluarkan dari stack
Elemen Teratas adalah : 20
Elemen pada stack :  20
                  10
```

Gambar 2.9

2.4.2. Penjelasan Program

Implementasi kelas `stackArray` menggunakan array statis berkapasitas 1000 dengan pointer `top` awal -1 untuk mengelola data secara efisien dengan kompleksitas $O(1)$. Melalui simulasi pengisian data (nilai 10, 20, 30), sistem menerapkan prinsip LIFO secara ketat, di mana eksekusi `pop()` menghapus elemen terakhir (30) dan `peek()` memverifikasi elemen di bawahnya (20) sebagai puncak baru. Konsistensi urutan data divalidasi oleh metode `print()` yang melakukan iterasi mundur untuk menampilkan data terbaru hingga terlama, sementara keamanan operasional tetap terjaga melalui penanganan kondisi overflow dan underflow yang komprehensif.

2.5 StackPostFix

2.5.1 Kode Program dan Output

```
StackPostFix_2511532008.java X
1 package pekan3_2511532008;
2 import java.util.*;
3
4 public class StackPostFix_2511532008 {
5     public static int postfixEvaluate(String expression) {
6         Stack<Integer> s = new Stack<Integer>();
7         Scanner input_2511532008 = new Scanner(expression);
8         while (input_2511532008.hasNext()) {
9             if (input_2511532008.hasNextInt()) {
10                 s.push(input_2511532008.nextInt());
11             } else {
12                 String operator_2511532008 = input_2511532008.next();
13                 int operand2 = s.pop();
14                 int operand1 = s.pop();
15                 if (operator_2511532008.equals("+")) {
16                     s.push(operand1 + operand2);
17                 } else if (operator_2511532008.equals("-")) {
18                     s.push(operand1 - operand2);
19                 } else if (operator_2511532008.equals("*")) {
20                     s.push(operand1 * operand2);
21                 } else {
22                     s.push(operand1 / operand2);
23                 }
24                 input_2511532008.close();
25                 return s.pop();
26             }
27         }
28         public static void main(String[] args) {
29             System.out.println("hasil postfix = "+postfixEvaluate("5 2 4 * + 7 -"));
30         }
31     }
```

Gambar 2.10

```
Console X
<terminated> StackPostFix_2511532008 [Java A
hasil postfix = 6
```

Gambar 2.11

2.5.2 Penjelasan Program

Implementasi kalkulator postfix ini menggunakan Scanner dengan metode `input.hasNext()` dan `input.hasNextInt()` untuk mem-parsing token secara dinamis, di mana setiap angka segera dimasukkan ke tumpukan melalui `s.push()`. Saat operator terdeteksi, program melakukan dua kali `pop()` untuk mengambil operand1 (puncak tumpukan) dan operand2 (elemen di bawahnya), kemudian menerapkan logika aritmetika dengan urutan krusial seperti `operand2 - operand1` atau `operand2 / operand1` guna menghindari kesalahan hasil pada operasi non-komutatif. Melalui struktur percabangan `if-else if`, setiap hasil perhitungan antara di-push kembali ke stack, sebagaimana ditunjukkan dalam simulasi ekspresi `5 2 4 * + 7 -` yang secara sistematis menghasilkan nilai akhir 6 berdasarkan prinsip LIFO.

BAB III

PENUTUP

3.1 Kesimpulan

Rangkaian praktikum ini membuktikan bahwa Stack dengan prinsip LIFO merupakan struktur data krusial untuk manajemen memori dan alur logika yang dapat diimplementasikan secara fleksibel. Melalui penguasaan metode fundamental seperti push, pop, dan peek, serta penerapan validasi isEmpty dan isFull, keamanan data terjaga dari risiko overflow maupun underflow. Lebih jauh lagi, penggunaan strategi backup stack dan algoritma evaluasi Postfix menunjukkan efisiensi Stack dalam menangani perhitungan kompleks, yang pada akhirnya memperkuat pemahaman mengenai mekanisme internal komputer seperti runtime stack untuk tantangan pemrograman tingkat lanjut.

DAFTAR PUSTAKA

W3Schools, “Java Tutorial,” 2026. [Daring]. Tersedia pada:
<https://www.w3schools.com/java/default.asp>.

TUGAS PRAKTIKUM

Class ADT Website

```
1 package pekan3_2511532008;
2
3 public class Website_2511532008 {
4
5     private String judul_2008;
6     private String url_2008;
7
8     public Website_2511532008(String judul_2008, String url_2008) {
9         this.judul_2008 = judul_2008;
10        this.url_2008 = url_2008;
11    }
12
13    public String getJudul_2008() {
14        return judul_2008;
15    }
16
17    public String getUrl_2008() {
18        return url_2008;
19    }
20
21    public void setJudul_2008(String judul_2008) {
22        this.judul_2008 = judul_2008;
23    }
24    public void setUrl_2008(String url_2008) {
25        this.url_2008 = url_2008;
26    }
27 }
28
```

Class Driver Browser

```
1 package pekan3_2511532008;
2 import java.util.*;
3 // Kelas Driver
4 public class Browser_2511532008 {
5     public static void main(String[] args) {
6         Stack<Website_2511532008> history = new Stack<>();
7         Scanner scanner = new Scanner(System.in);
8         int pilihan = 0;
9         do {System.out.println("=== Browser History NIM: 2511532008 ===");
10             System.out.println("1. Kunjungi Website (Push)");
11             System.out.println("2. Tombol Back (Pop)");
12             System.out.println("3. Lihat Halaman Aktif (Peek)");
13             System.out.println("4. Cek Status History");
14             System.out.println("5. Keluar");
15             System.out.print("Pilihan: ");
16             if (scanner.hasNextInt()) { pilihan = scanner.nextInt(); scanner.nextLine(); }
17             else { System.out.println("Input tidak valid. Masukkan angka.");
18                 scanner.nextLine();
19                 continue; }
20             switch (pilihan) {
21                 case 1:
22                     System.out.print("Masukkan Judul: ");
23                     String judul = scanner.nextLine();
24                     System.out.print("Masukkan URL: ");
25                     String url = scanner.nextLine();
26                     history.push(new Website_2511532008(judul, url));
27                     System.out.println("Berhasil mengunjungi halaman!");
28                     break;
29                 case 2:
30                     if (history.isEmpty()) {
31                         System.out.println("History kosong! Tidak ada halaman untuk di-back.");
32                     } else { Website_2511532008 removedPage = history.pop();
33                         System.out.println("Berhasil Back! Meninggalkan halaman: " + removedPage.getJudul_2008());
34                     }
35                     break;
36                 case 3:
37                     if (history.isEmpty()) { System.out.println("Tidak ada halaman yang sedang aktif (History kosong).");
38                     } else { Website_2511532008 currentPage = history.peek();
39                         System.out.println("=== Halaman Aktif Saat Ini ===");
40                         System.out.println("Judul : " + currentPage.getJudul_2008());
41                         System.out.println("URL : " + currentPage.getUrl_2008()); }
42                     break;
43                 case 4:
44                     if (history.isEmpty()) {System.out.println("Status: Riwayat browser saat ini kosong.");
45                     } else {System.out.println("Status: Terdapat " + history.size() + " halaman tersimpan dalam riwayat.");
46                     }
47                     break;
48                 case 5:
49                     System.out.println("Keluar dari browser. Terima kasih!");
50                     break;
51                 default:
52                     System.out.println("Pilihan tidak tersedia. Silakan coba lagi.");
53             }
54         } while (pilihan != 5);
55         scanner.close();
56     }
```